

Project Manager Simulation Environment

Goal

Build a working simulation environment for a project manager's first week at a small-to-medium SaaS company.

The environment should support the core loop of project management work:

- time advances through a simulated work week
- the agent interacts through internal tools such as chat, email, calendar, task tracking, and docs
- coworkers have their own goals, constraints, and background activity
- the agent must discover blockers, resolve conflicts, prioritize tradeoffs, and keep projects moving
- the system evaluates whether the agent's actions actually improved outcomes

We are intentionally not prescribing the architecture. You should make the core decisions yourself: how simulated time advances, how world state is stored, how events are scheduled, how tool surfaces are exposed, how NPC behavior is modeled, how evaluation works, and what the operator workflow looks like.

We are much more interested in the systems problems behind a realistic simulation than in surface-level mimicry: state transitions, event delivery, discoverability of information, long-horizon consistency, and defensible grading. This does not need to be distributed. A strong single-node design with clear semantics is better than a needlessly decomposed system.

We care about whether you have strong, defensible opinions about how to build and operate a realistic environment from core principles. Coding agents are welcome as part of the workflow, but the design decisions should still be clearly yours and should be legible in the final system.

Constraints

- The deliverable must be a **single repository**
 - A reviewer must be able to clone the repo, start the full system locally, and exercise the main flows using documented commands
-

Scope

You should build enough of the environment that the major design choices become concrete.

At minimum, the system should demonstrate:

- explicitly simulated time that is decoupled from wall-clock inference latency
- a persistent company and project world model with at least one active project containing dependencies, blockers, and stakeholder pressure
- integrated tool surfaces like chat, email, calendar, meeting/transcript capture, task tracking, and document management
- multiple stateful coworkers with distinct roles and behavior, including proactive outreach and realistic response delays
- evaluation that rewards improved outcomes and sound decisions more than superficial activity

Your implementation should make your systems choices legible. We should be able to tell what advances synchronously with an agent action, what advances asynchronously in the background, how scenario state is owned and mutated, how evaluation is computed, and how the system would grow to support many more scenarios without collapsing into prompt spaghetti or hand-authored one-offs.

If you use model-based verification anywhere, be explicit about why it is needed, what inputs it sees, and how you keep results stable enough to trust.

Beyond that, scope is up to you. You can decide how much to invest in UI, APIs, operator tooling, observability, scenario authoring, and NPC sophistication. A lightweight interface is fine. We care more about coherence and realism than about polish.

Deliverables

Your repository should include:

- the full runnable system
- at least one fully-authored project manager scenario with seeded company state, tool data, coworker personas, and evaluation ground truth
- a `README.md` with setup, how to start it, how to drive the main flows, and how to run the evaluation
- enough detail on grading that a reviewer can understand the score components, inspect example outcomes, and see how the evaluator resists reward hacking
- enough documentation for a reviewer to understand your architecture and realism choices

